VentureMiner AI

Coding Standards (Web & Frontend)

Document: 08-Engineering / 27_coding_standards_web
Version: v1.0
Date: 2026-07-20
Author: VentureMiner AI Documentation Team
Status: Approved
Project: VentureMiner AI — AI Venture Intelligence Platform

Table of Contents

Document 27 — Coding Standards (Web & Frontend)

The companion to Document 26, focused on the web app and frontend-specific practices. Where this conflicts with Document 26, this document wins.

Table of Contents

1. Purpose & Scope

This document is the contract for the web app (apps/web). It assumes Document 26 (general coding standards) and adds the frontend-specific rules.

2. Framework & libraries

- Next.js 15 (App Router, RSC where useful).
- React 19.
- TypeScript 5 (strict).
- Tailwind CSS for styling.
- Radix UI for primitives.
- React Query for server state.
- Zustand for UI state.
- Zod for validation.
- React Hook Form for forms.

- Vitest for unit.
- axe-core for a11y tests.

3. Component model

- One component per file, named the same as the file.
- Server Component by default; Client Component with "use client" only when needed.
- Composition over inheritance.
- Props are typed with interface (or type for unions).
- Children are preferred over renderX props.
- No default exports for components (named exports).

3.1 Component anatomy

```
"use client"; // only if needed

import { useState } from "react";
import { Button } from "@/components/button";

export interface OpportunityCardProps {
opportunity: Opportunity;
onSelect?: (id: string) => void;
}

export function OpportunityCard({ opportunity, onSelect }: OpportunityCardProps) {
// ...
}
```

4. State management

- Server state → React Query.
- UI state → Zustand.
- Form state → React Hook Form.
- URL state → query params.
- Local state → useState for trivial cases.
- No Redux. No Context-as-store.

5. Data fetching

- React Query for all API calls.
- Mutations invalidate the relevant queries.

- Error handling at the query level (toast on error).

6. Forms

- React Hook Form + Zod.
- Schema in a separate file; reusable.
- Validation messages in the i18n catalog.
- Submit buttons disable on submit; show loading state.
- Errors display below the field with aria-describedby.

7. Routing

- App Router with nested layouts.
- Dynamic routes use [id] convention.
- Loaders for prefetching.
- Error boundaries per route segment.
- Loading.tsx for each segment.

8. Styling

- Tailwind for everything.
- No inline styles.
- Design tokens in tailwind.config.ts — no raw hex in components.
- clsx for conditional classes.
- Variants via cva (class-variance-authority).

9. Accessibility in code

- Semantic HTML first.
- ARIA only when semantics are insufficient.
- Focus management explicit; no tabindex > 0.
- Keyboard handling per WAI-ARIA Authoring Practices.
- Color contrast verified in Storybook (a11y addon).
- Reduced motion respected.

10. Performance

- Code-split per route.
- Lazy load below-the-fold images.

- Virtualize long lists (@tanstack/react-virtual).

11. Testing

- Unit: Vitest + React Testing Library.
- Component: Storybook + a11y addon.
- Visual regression: Percy.
- E2E: Playwright.
- A11y: axe-core in E2E.

12. Storybook

- Every public component has a story.
- Stories cover default, all variants, all states.
- A11y addon enabled; warnings fail CI.
- Visual regression on every PR.

13. Internationalization

- All strings via t('...').
- No hard-coded text in components.
- Pluralization via ICU message format.
- Date/number/currency via Intl.

14. Error boundaries

- Per route segment to localize failures.
- Fallback is a friendly error UI with a retry CTA.
- Errors logged with console.error + sent to Datadog.
- Sentry-style source maps for production.

15. Appendix

15.1 Revision history

v0.5	2026-07-20	Doc Team	All sections drafted
v1.0	2026-07-20	Doc Team	First approved version

15.2 Cross-references

- Coding Standards: Document 26.

End of Document 27 — Coding Standards (Web & Frontend).